

Report TP1

Baptiste DESLOGE

Selma ELMSETEF

APP5 IIM

Part 1 – Baseline model

Architecture :

In our architecture we got 2 hidden layers. First one is of dimension 256 and the second one is of dimension 128. We work with image of 28x28 dimensions which mean 784 pixels. We got 10 different categories which means 10 output logits.

Architecture : $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$

Total parameters numbers :

$784 \times 256 + 256 \text{ (bias)} = 200,960$

$256 \times 128 + 128 = 32,896$

$128 \times 10 + 10 = 1,290$

Total parameters = 235,146

Loss function :

Since we work on a classification problem, the appropriate loss function is the CrossEntropyLoss. Indeed, it is specifically designed to compare probabilities rather than absolute distances. Since a classification model outputs the percentage chance of a data point belonging to a certain category, this function measures how far off those predictions are from the actual labels.

Training configuration :

We chose the Adam optimizer because, as stated in CM2, it is the standard base optimizer. We set the learning rate to $1e-3$ as it is the default for Adam, providing efficient learning. We didn't use regularization to see how the model naturally overfits the training data. Finally, we train for 15 epochs because this is sufficient time to observe the model's convergence.

Baseline results :

On the loss graph we can see that the training loss decreases continuously, but the validation loss stops improving around epoch 8 and begins to slowly increase.

For the accuracy graph we observe that the validation accuracy stagnates around epoch 8, while the training accuracy continues to rise. At the final epoch, the training accuracy reaches approximately 93% and the validation accuracy converges to 88.5%.

Finally, when we look at the last graph, we see that the accuracy gap is about 4.4%. It progressively increases. Based on it and the loss graph we can assert that the baseline model is overfitting the training data.

Experiment 1 - Model Architecture

Config	Train acc (ep 15)	Val acc (ep 15)	Gap	Test acc
Micro [4]	82,94%	82,18%	1%	81,55%
Small [64]	92,22%	88,06%	4%	87,63%
Baseline [265,128]	94,04%	89,51%	4,5%	88,87%
Large [512, 256]	94,26%	89,11%	5%	89,03%

Underfitting

We can observe that with the micro config, the train accuracy curve is very low compare to the other config. It stagnates very early, struggling to reach approximately 83%, whereas the other architectures exceed 90%. However, the train-validation accuracy gap is exceptionally small, remaining almost flat after the first epoch. A small gap does not guarantee good generalization, it means the model performs equally on both the training and validation data. In the case of the Micro config, the gap is small because the network lacks the structural capacity to learn the data's underlying patterns. Consequently, it performs poorly on both datasets. This why it's an underfitting.

Overfitting

For all the other model (small, baseline, large), we can observe a wide gap over the 15 epochs.

The trajectory follows a similar pattern, first the gap starts in the negative range where validation accuracy is briefly higher than training accuracy. After the second epoch, the gap grows in a linear way. The rate of this widening correlates with the size of the network. The large architecture show the widest gap, followed by the baseline and then the Small architecture. This upward trajectory indicates that as training progresses, these models with higher capacity are increasingly memorizing the training data rather than generalizing. This is an overfitting.

Val accuracy plateau

Based on the validation accuracy graph, we can see that the validation accuracy begins to plateau at the small capacity. Under the current training setup, we can conclude that adding more capacity beyond the small config provides no meaningful benefit for generalization. The extra parameters in the baseline and large models are not helping the network learn better underlying patterns. They are increasing the model's tendency to overfit the training data.

Bias-variance trade-off

Based on the gap results, we can assert that as the model capacity increases, the final train-validation accuracy gap strictly increases.

Architecture selection

Based on all these observations, the most relevant architecture seems to be the small one. Indeed, we achieve a precision on the validation data of 88% compared to 89% for the largest. The gap between the accuracy with this configuration and the more complex ones is insignificant. This configuration therefore allows for a good compromise between precision and complexity. With it, we have only one hidden layer and we limit overfitting.

Experiment 2 - Optimiser and Learning Rate

2.A Learning rate

LR	Val acc (15 ep)	Test acc (15 ep)
1e-5	21,23%	20,71%
1e-3	81,64%	80,48%
0.1	89,12%	88,09%

Convergence behavior :

For $lr=0.1$: The training loss drops sharply in the first few epochs, and the validation accuracy immediately jumps to approximately 85%, plateauing near 90% very early. This indicates that the step size is sufficient to quickly navigate the loss landscape and maintain a good minimum.

For $lr=1e-3$: The loss decreases continuously, and validation accuracy climbs over 80%. It is learning effectively but slower, meaning it would need significantly more than 15 epochs to fully converge.

For $lr=1e-5$: The model completely struggles to learn. The training loss remains almost entirely flat, and the validation curve barely moves upward. After 15 epochs, the model is severely underfitting. Because the learning rate is too small, the network's weights are barely updating during backpropagation. It results in a stagnant training loss and a final validation accuracy of only 20%.

The model with a learning rate of 0.1 converges rapidly to approximately 85% accuracy in the first epoch and stagnate near 90%, whereas the model with a $1e-3$ learning rate progresses slowly, starting around 60% and reach 80% by epoch 15. This highlights the high cost of a suboptimal learning rate choice. Specifically, a learning rate that is too small wastes valuable computational resources and time by requiring significantly more epochs to reach peak performance. It could lead to bad model evaluations, as you might discard a good neural network architecture simply because it was paired with a slow optimizer and didn't finish learning within a limited budget of epochs.

2.B Optimiser comparison

Config	Learning rate	Best val acc	Epochs to reach best
SGD best	0.1	89.38%	9
Adam best	$1e-3$	89.22%	15

The highest validation accuracy is achieve with a learning rate of 0.1 for SGD optimizer and $1e-3$ for adam optimizer.

If we compare their best configurations, we can see that both optimizers exhibit similar convergence trajectories. SGD reach its best validation accuracy at epoch 9 compared to Adam at epoch 15. This suggests that, although Adam's adaptive parameter updates are intended to accelerate the learning process, a properly tuned standard SGD can achieve a comparable convergence rate.

When both optimizers use learning rate of $1e-3$, we observe that Adam converges rapidly, while SGD progresses slowly and finish with lower accuracy. This show that optimal hyperparameters are specific to the underlying algorithm and that applying same global learning rate across different optimizers will lead to different training dynamics.

Experiment 3 – Regularisation

Config	Train (ep20)	Val (ep20)	Gap	Test acc	Epoch of best val acc
No regularisation	95.33%	88.98%	6.2%	88.46%	19
Dropout 0.3	90.58%	89.2%	1.6%	88.49%	17
Weight decay $1e-3$	90.65%	88.07%	2.2%	87.42%	16
Dropout + BN	91.57%	89.78%	1.8%	89.46%	20

Dropout prevents overfitting by randomly deactivating some neurons during training to stop them from adapting. Weight decay adds a penalty to the loss function to keep the model's weights small and limit its overall complexity.

Looking at the results, the configuration utilizing Dropout achieves the smallest train-validation accuracy gap (1.6%). The combination of Dropout and Batch Normalization achieves the highest overall validation accuracy of 89.78%.

Looking at the training dynamics, regularization techniques help the model reach its best performance in fewer epochs. Without regularization, the highest validation accuracy is reached at epoch 19, compared with epoch 16 when using weight decay and epoch 17 with dropout. In contrast, combining dropout with batch normalization seems to slow down the training process, with the best validation accuracy being reached at epoch 20. These results indicate that simple regularization techniques can help the model generalize more quickly by reducing early overfitting. However, if we use several regularization methods together, it can change the learning dynamics and may require more training time for the model to converge.